

操作手冊

資訊所碩一 P76121314 鄭煜醴

Server 建置(包括 api)：

1. 下載所有需要的套件(pip install requirements.txt)
2. 建立 Django 項目(django-admin startproject <專案名稱>)
3. 進入專案資料夾 (cd <專案名稱>)
4. 創建 api 資料夾(python manage.py startapp <my_app>)
以下依據我專案(Django 項目)的資料夾名稱來演示
5. 建立 LLM 模型(api/LLM_Model.py)
 - i. 模型採用 Chat-GLM-6B 架構(以 transformer 為框架)
 - ii. 模型結構位址(code\chatglm-6b\modeling_chatglm.py)
 - iii. 參考出處：<https://github.com/THUDM/ChatGLM-6B/tree/main>
6. 在 api/views.py 建立 view(負責處理用戶請求並返回 response)
7. 在 api/urls.py 中配置 URL(將 URL 連接到 view)
8. 設置 my_server/urls.py(負責定義 URL 與 views 之間的映射關係) 註:api/urls.py 內的 URL 是其子集(包含在 myserver/urls.py 裡面)
9. 在 myserver/settings.py 中設置 ALLOWED_HOSTS = ['*'](表示該 Django application 可以接受任何主機的請求)
10. 在 my_server/settings.py 中 INSTALLED_APPS 加入紅框項目

```
INSTALLED_APPS = [  
    "django.contrib.admin",  
    "django.contrib.auth",  
    "django.contrib.contenttypes",  
    "django.contrib.sessions",  
    "django.contrib.messages",  
    "django.contrib.staticfiles",  
    "api.apps.ApiConfig",  
    'rest_framework'  
]
```

11. 在 api/apps.py 中呼叫載入權重模型的 function(api/load_model.py)使 server 一啟動時會先 load 模型權重。

```
class ApiConfig(AppConfig):
    default_auto_field = "django.db.models.BigAutoField"
    name = "api"

    tokenizer = None
    model = None

    def ready(self):
        load_weight()
        print("Model and tokenizer have been loaded")
```

```
from transformers import AutoTokenizer, AutoModel

model = None
tokenizer = None

def load_weight():
    global model, tokenizer
    if model is None or tokenizer is None:
        model_identifier = "chatglm-6b"
        tokenizer = AutoTokenizer.from_pretrained("chatglm-6b", trust_remote_code=True)
        model = AutoModel.from_pretrained("chatglm-6b", trust_remote_code=True)
        print("權重下載完畢")
```

直接在本機端啟動 server(方案 1)：

建立好環境後(requirements.txt)，在 manage.py 同層目錄輸入命令(python manage.py runserver 0.0.0.0:8000 --noreload)即可啟動 Django server

Docker 建置 server(方案 2)：

直接在 code 目錄下執行 bash start.sh 即可建置 image 和啟動 server，並可直接測試。

注意：建置 docker image 到啟動 server 並加載 LLM 模型(需要大概 30 分鐘)，加載完即可正常使用 LLM 模型 API。

遇到問題：在 docker 上啟動伺服器時會遇到 memory 內存不足問題導致無法運行。

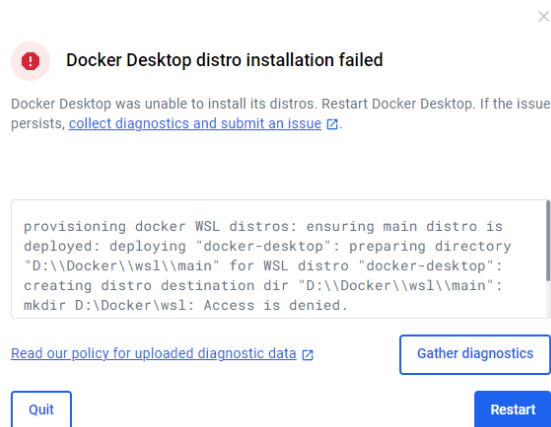
解決方法：將 docker 改為主要在 disk 中運行以達到減少 memory 內存的消耗。步驟如下：

1. 刪除現有的 docker desktop
2. 下載 docker desktop exe 檔(下載完後先不要安裝)
3. 在 docker desktop exe 檔同目錄下開啟 cmd 系統管理員
4. 輸入以下指令安裝 docker 到 disk(D 槽)：

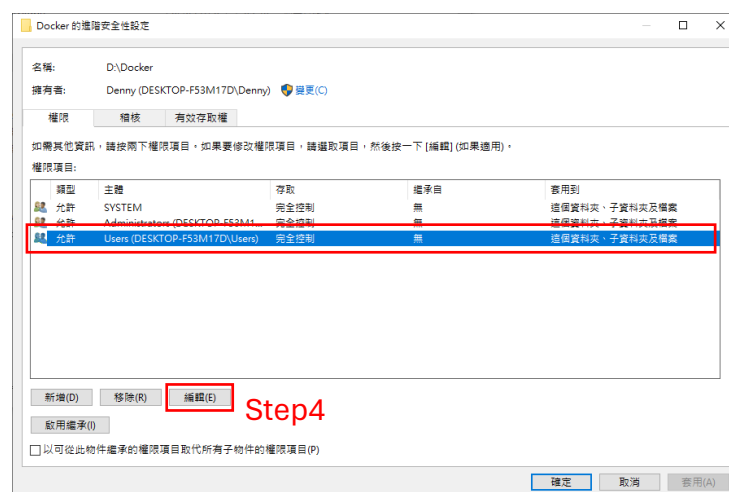
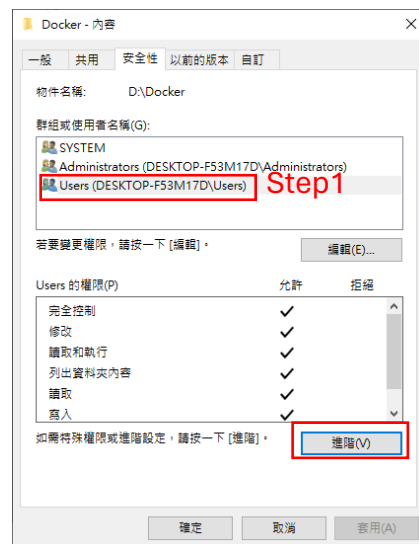
```
start /w "" "Docker Desktop Installer.exe" install -accept-license --
installation-dir="D:\Docker\Docker" --wsl-default-data-
root="D:\Docker\wsl" --windows-containers-default-data-
root="D:\Docker"
```

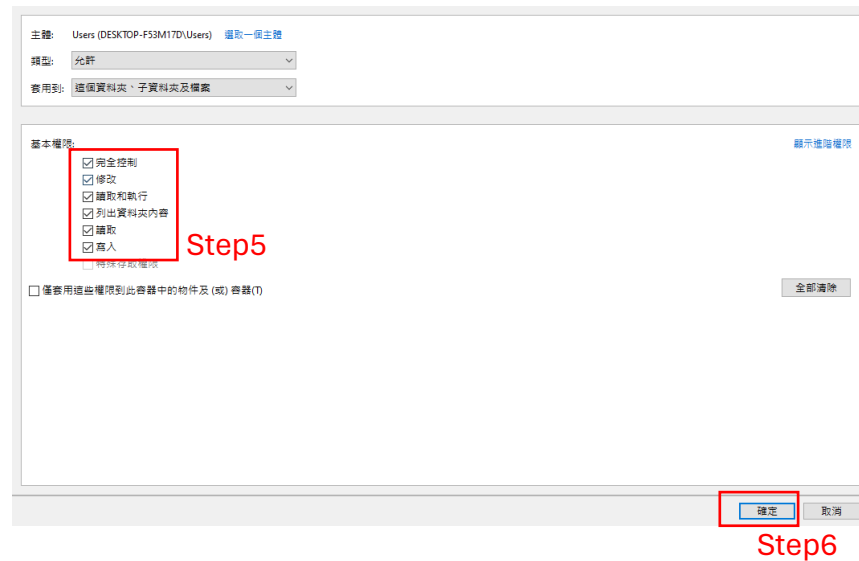
```
C:\Users\Denny\Downloads>start /w "" "Docker Desktop Installer.exe" install -accept-license --installation-dir="D:\Docker\Docker" --wsl-default-data-root="D:\Docker\wsl" --windows-containers-default-data-root="D:\Docker"
```

5. 安裝完後打開 docker desktop(出現該畫面關閉即可)

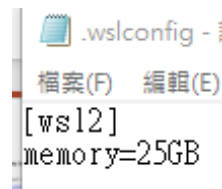


6. 找到 D 槽根目錄下的 Docker 目錄並點擊右鍵進入內容，至安全性修改 user 權限(全開)，設置完成點選確定。





7. 重新開機
8. 至 user 目錄(ex：C:\Users\Denny)下建立.wslconfig 檔案並輸入內容



9. 之後再 cmd 系統管理員輸入 wsl --shutdown 終止原預設的 wsl，之後輸入 wsl 重新啟動由.wslconfig 設置的 wsl

```
Microsoft Windows [版本 10.0.19045.4239]
(c) Microsoft Corporation. 著作權所有，並保留一切權利。
C:\Windows\system32>wsl --shutdown
C:\Windows\system32>wsl
(base) denny@DESKTOP-F53M17D:/mnt/c/Windows/system32$
```

10. 以上步驟做完後重新啟動 server 即可順利啟動 server

系統測試 1(在終端機直接輸入 prompt)：

進入 code 目錄執行 start.sh 後等 server 啟動後，即可直接輸入 prompt，並得到 LLM 模型的 response(可能要稍等一下子)，輸入 exit 或 ctrl+C 即可退出。

以下為輸入流程：

Input 為 prompt 的文字，main.py 會在用戶輸入後將 prompt 轉成 json 格式(ex：{"prompt": "Input"})並發送 POST 請求將 prompt_json 傳給 api，之

後 api 處理完 prompt 後會再將輸出返還給用戶 (ex : {“status”:True,
‘data’: {“response”: “output”}})

系統測試 2(在網頁上輸入 prompt)：

可連至下列網址「127.0.0.1:8000/api/chat/」即可通往此頁面

進入頁面後在 Content 輸入 prompt (注意：輸入格式為 {“prompt”: “您的輸入”}
必須為雙引號 範例如下)

The screenshot displays the Llm API testing interface. At the top, there's a header with 'Llm' and an 'OPTIONS' button. Below it, a text box shows a GET request to '/api/chat/' which resulted in a 405 Method Not Allowed error. The error message is: 'HTTP 405 Method Not Allowed', 'Allow: POST, OPTIONS', 'Content-Type: application/json', 'Vary: Accept', and a JSON object with 'detail': 'Method \"/api/chat/'.

Below the error message, there's a form for making a POST request. The 'Media type' is set to 'application/json'. The 'Content' field contains the JSON object: {“prompt”: “你知道台灣首都在哪裡嗎”}. This field is highlighted with a red box and labeled '輸入' (Input). To the right of the 'Content' field, there's a button labeled 'POST' and a tooltip that says 'Make a POST request on the Llm resource'.

Below the form, there's a text box showing the response to the POST request. The response is: 'HTTP 200 OK', 'Allow: POST, OPTIONS', 'Content-Type: application/json', 'Vary: Accept', and a JSON object: {“status”: true, “data”: {“response”: “台灣的首都是台北市。”}}. This response is highlighted with a red box and labeled 'response'.

輸入後即可獲得 response。

目錄展示：

